This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

A 28-nm Sign-Bit-Embedded Bit-Parallel SRAM Compute-in-Memory Macro With Stage-Wise-Enabled Transition-Counting-Lines and Accumulators for Edge-AI Devices

Yunlong Liu, *Graduate Student Member, IEEE*, Lichen Feng, Lin Wu, Dong Li, and Zhangming Zhu

***Abstract*—Computing-in-memory (CIM) architectures are promising for edge-AI devices, as they mitigate the von Neumann bottleneck and reduce data movement. Bit-parallel CIM macros with direct memory mapping and low toggle rate are well-suited for computing systems. However, severe routing congestion, inefficient signed computation, and unnecessary toggling of intermediate signals in local multiply-accumulate (MAC) circuits remain unsolved. This article presents a 28-nm bit-parallel SRAM CIM macro for efficient INT8 and BF16 MAC operations, featuring three techniques: 1) a transition-counting-line (TCL) that performs sparsity-adaptive, lossless bit-column compression by recording full-swing transitions derived from partial products, thereby mitigating routing congestion while preserving full precision; 2) a bus-based TCL that reuses sign-bit multiply-multiplex units (MMUs) to realize embedded sign extension with only 3% area overhead and no additional cycles; and 3) a shared ten-phase generator that enables TCLs and accumulations in a stage-wise manner to suppress spurious toggling, improving energy efficiency by 42%. Fabricated in 28-nm CMOS, the 32-kb macro occupies 0.072 mm^2 and supports 64 input channels. The macro achieves a 4.1-ns access time at 0.9 V, with peak energy efficiencies of 106.85 TOPS/W in INT8 mode and 77.68 TFLOPS/W in BF16 mode, and peak area efficiencies of 1.73 TOPS/mm^2 and 1.24 TFLOPS/mm^2, respectively.**

***Index Terms*—Artificial intelligence (AI), bit parallel, computing in memory (CIM), lossless bit-column compression, transition counting line (TCL).**

I. Introduction

With the rapid advancement of artificial intelligence (AI), neural-network models have expanded dramatically in both size and complexity [1], [2]. However, conventional von Neumann architectures require frequent data movement between memory and processing units, resulting in substantial energy consumption and latency. Computing-in-memory (CIM) architecture has emerged as a promising solution to address this challenge by integrating memory arrays with local computing units to improve energy efficiency and to reduce the latency of multiply-accumulate (MAC) operations [3].

Implemented with either volatile SRAM [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], eDRAM [32], [33], or nonvolatile RRAM [34], MRAM [35], [36], and Flash [37], [38], etc., CIM has been extended to support a wide range of numerical formats, including INT4 [20], [21], INT8 [14], [30], and floating-point formats such as FP16/BF16 [27], [29], MXFP [12], [39], and POSIT [40], [41].

To realize multi-bit MAC efficiently, bit-serial CIM has been widely adopted in prior works [4], [6], [11], [20], [21], [27], [28], [42]. A multi-bit MAC operation is decomposed into single/dual-bit operations over multiple cycles, thereby avoiding bulky parallel multipliers and high-fan-in adder trees. Bit-serial facilitates lightweight floating-point manipulation and can exploit bit-level sparsity to reduce power consumption [6], [23]. However, as shown in Fig. 1, as CIM macros are integrated into practical neural network accelerators [27], [33], [43], [44], [45], [46], [47], [48], bit-serial operation leads to inefficiencies. First, practical neural-network accelerators natively process multi-bit data in parallel, whereas bit-serial CIM interacts with the system through serialized single/dual-bit inputs, requiring additional parallel-to-serial conversion and buffering/data-reformatting overhead. Second, serialized bit-level computation typically requires customized data organization such as weight slicing across memory rows, columns, or cycles [11], [42]. Third, prior works [9], [49] have shown that bit-parallel CIM has a lower toggle rate than bit-serial CIM across various network models, since the most significant bits (MSBs) of adjacent inputs have strong correlation, whereas extra data-representation conversion [11], [13] is required to suppress switching activity in bit-serial CIM. For these reasons, bit-parallel CIM is becoming increasingly attractive [9], [14], [15], [49].

Nevertheless, as summarized in Fig. 2, bit-parallel CIM still faces several critical challenges when implemented

Received 6 May 2026; revised 8 July 2026; accepted 27 July 2026. This article was approved by Associate Editor Massimo Alioto. This work was supported in part by the National Key Research and Development Program of China under Grant 2022ZD0118903; and in part by the National Natural Science Foundation of China under Grant 62474128, Grant U22A2013, and Grant U24A20291. *(Corresponding author: Lichen Feng.)*

Yunlong Liu, Lichen Feng, Lin Wu, and Dong Li are with the Key Laboratory of Analog Integrated Circuits and Systems, Ministry of Education, School of Integrated Circuits, Xidian University, Xi'an 710071, China (e-mail: lcfeng@xidian.edu.cn).

Zhangming Zhu is with Zhejiang Key Laboratory of Analog Integrated Circuits, Hangzhou Institute of Technology, Hangzhou 311231, China and also with the School of Integrated Circuits, Xidian University, Hangzhou 311231, China.

Digital Object Identifier 10.1109/JSSC.2026.3718075

0018-9200 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:18:21 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

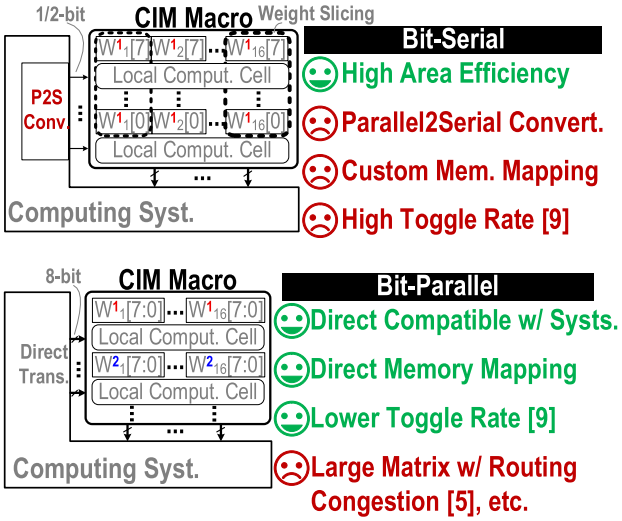


Fig. 1. Advantages and disadvantages of bit-serial and bit-parallel CIMs, respectively.

under a strict SRAM-pitch-matched local-computing-unit constraint.

The first physical-design challenge is local routing congestion. In a direct digital 8-b ×8-b bit-parallel implementation, 64 one-bit partial products are generated simultaneously and must be accumulated by a local adder tree. To preserve memory density and macro regularity, the local computing unit is required to match the height of the corresponding SRAM rows, which results in severe routing congestion in digital CIM in terms of physical-layout constraints [5], [9]. Existing approaches either concatenate lower-bit-width MACs across multiple cycles at the cost of reduced throughput [15], [30], or employ analog bit-column compression with ADC overhead [10], [32].

The second challenge is that two's-complement signed MAC requires sign-bit extension, which introduces additional sign-extension partial products and increases routing and switching overhead. Sign-bit handling inside the multiplier improves single-cycle capability but incurs area overhead [15], [30], whereas handling after multiplication reduces in-array complexity but typically requires additional cycles [27].

The third challenge is spurious toggling in adder trees. During multiplier settling, numerous intermediate transitions propagate through multistage adders and waste power [31]. Adder trees consume more than 59% of the total energy for 128 8-bit MAC in a bit-parallel CIM [49].

To address the above challenges, this work presents an energy- and area-efficient bit-parallel CIM macro. The primary contributions of this work are listed as follows.

Sparsity-adaptive transition-counting-line (TCL) is proposed to realize lossless bit-column compression by counting the full-swing transitions converted from partial products. Sixty-four partial products are compressed into 15 bit-column signals, reducing the local routing demand by 4.27× while preserving full-precision computation. In our 28-nm layout, the average available local-routing budget per signal is increased from 0.113 to 0.48 μm.

A bus TCL is introduced to support embedded signed two's-complement MAC by reusing sign-bit partial products in existing TCLs. As a result, signed computation is supported with only 3% extra area overhead and no additional cycles.

A shared local ten-phase generator is designed to orchestrate TCL and accumulation operations in a stage-wise manner, suppressing unnecessary signal propagation during TCL settling and improving overall energy efficiency by 42%.

The remainder of this article is organized as follows. Section II describes the overall architecture of the proposed CIM macro. Section III presents the detailed computing circuits. Section IV introduces the bus TCL and the stage-wise adder tree. Section V reports the experimental results. Finally, Section VI concludes this article.

II. Overall Architecture of TCL-Based CIM Macro

Fig. 3 shows that the proposed CIM macro consists of a global controller (Ctrl.), an SRAM read/write controller, four CIM banks, a global input buffering unit, and an exponent post-processing unit for BF16 operation. Each CIM bank includes a local ten-phase generator, an out-block adder tree, and four CIM blocks, where each CIM block contains 16 CIM cells and an in-block adder tree. The in-block adder tree sums the 16 products generated by the 16 CIM cells, while the out-block adder tree accumulates the four outputs from the four in-block adder trees to produce a 22-bit MAC value (MACV). Each CIM block integrates a 16 × 128b 6T-SRAM array for storing unsliced weights. To reduce area and power overhead, one local ten-phase generator is shared by four CIM blocks. Sixty-four input features (XIN in Fig. 3) are broadcast in parallel to the four CIM banks. Hence, the proposed CIM macro generates four 22-bit MACVs per clock cycle. The global input buffering unit further includes an exponent pre-alignment module for BF16 inputs.

In INT8 mode, the weights are stored in either signed two's-complement or unsigned format. In BF16 mode, the pre-aligned [4] BF16 weight significand {Sign, 1, Mantissa} is converted off-chip into a signed two's-complement format.

As shown in Fig. 4, the computing cell consists of 64 multiply-multiplex units (MMUs), 15 counters (CNTs), and one combine adder. In each MAC cycle, the computing cell reads one 8-bit weight word, i.e., one row of the 16 × 8b SRAM array. MMUs corresponding to the same bit position are hierarchically connected, with one CNT placed at the end of each hierarchy to form a TCL for bit-column addition. Accordingly, fifteen TCLs are designed for 8-bit × 8-bit multiplication, and the resulting 15 bit-column sums are accumulated by the combine adder to generate one 16-bit product. The MMUs associated with the sign bits in the bus TCL (TCL-14) are reused across TCL-7 to TCL-14 to realize embedded sign-bit extension for two's-complement multiplication. The local ten-phase generator produces ten clock phases, denoted as Ph0–Ph9, where Ph0–Ph7 sequentially enable the MMUs in each TCL, Ph8 enables the combine adder, and Ph9 enables the in-block adder tree.

As shown in Fig. 5, the proposed CIM macro supports both INT8 and BF16 MAC operations. The read

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:18:21 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

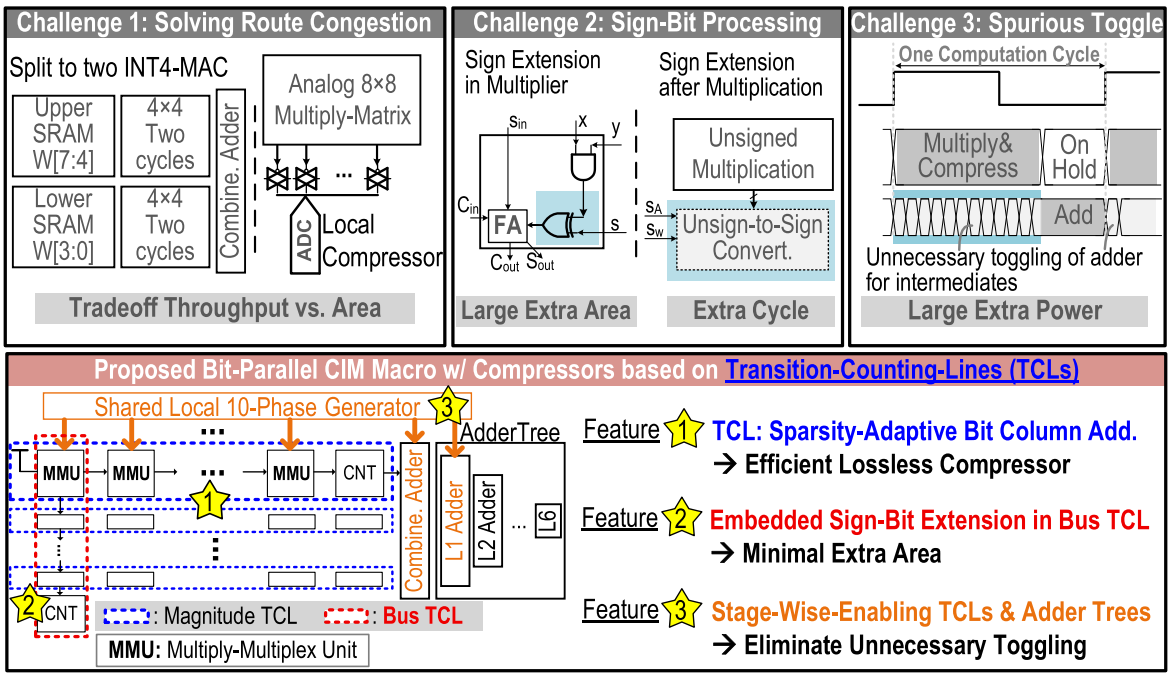


Fig. 2. Challenges of bit-parallel CIM macro and the proposed solutions.

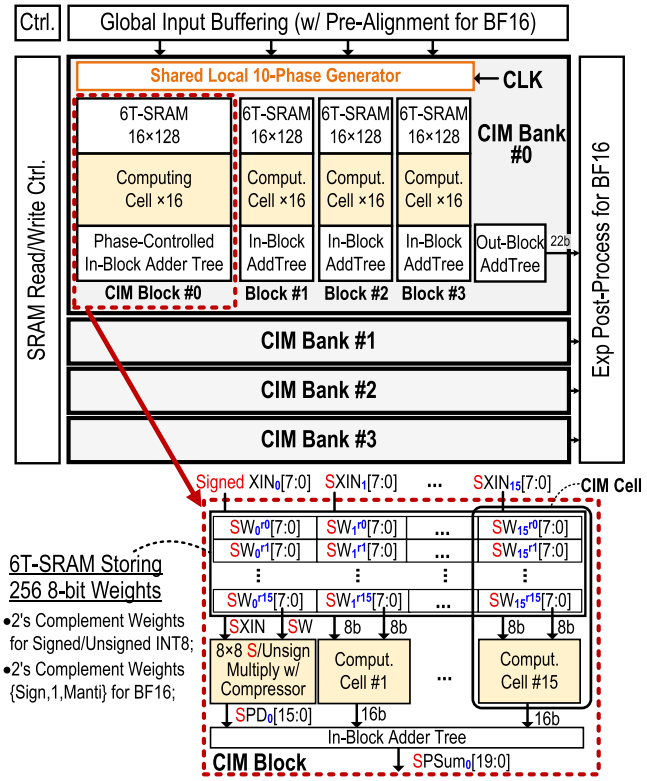


Fig. 3. Overall architecture of the proposed TCL-based digital-SRAM CIM macro.

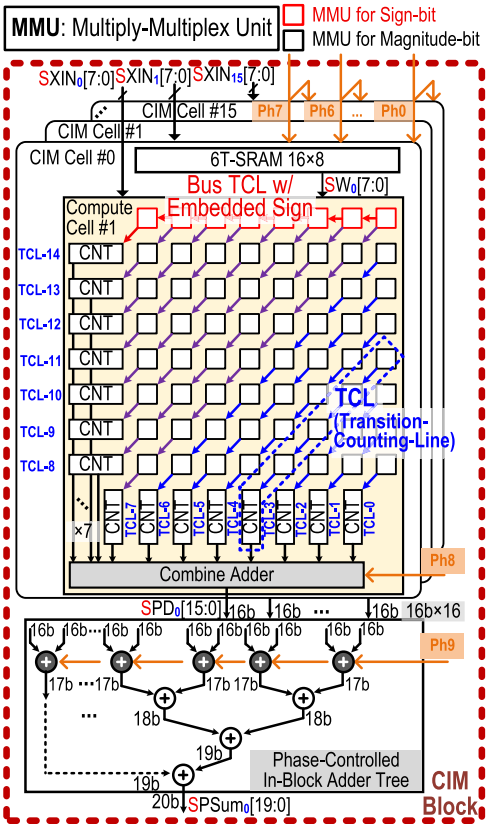


Fig. 4. Circuit structure of a CIM block consisting of 16 CIM cells and an in-block adder tree.

of weights through the sense amplifier occupies a clock cycle before a group of MAC operations. After this clock cycle, the weights are held for multiple MAC operations. In INT8 mode, the computation is completed within one clock cycle. In BF16 mode, a three-stage pipeline, including feature pre-alignment, MAC, and post-processing, is formed, generating FP32 output MACVs in every clock cycle after pipeline filling. The BF16 exponent pre-alignment and post-processing

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:18:21 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

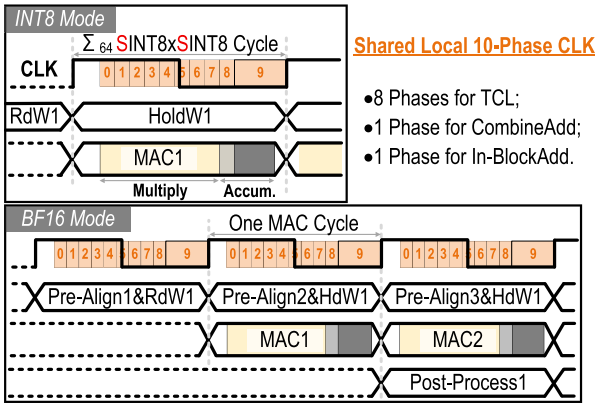


Fig. 5. Timing diagrams of INT8 and BF16 operations.

circuits are fully implemented as digital logic, described at the RTL level and realized through standard-cell-based synthesis and physical design. Within each system clock cycle, the local ten-phase generator produces ten local clock phases to control the operating sequence of the computing cells and the in-block adder tree.

III. Circuit Design of the TCL-Based CIM Macro

In this section, the circuit structure of the proposed TCL-based computing cell is first introduced (Section III-A, Fig. 6), followed by an illustrative bit-column addition example (Fig. 7) to facilitate understanding of its operating principle. The detailed timing analysis is presented in Section III-B, and the implementation details of the dual-edge counter are described in Section III-C. Section III-D shows an end-to-end multiplication example.

A. Circuit Structure and Operating Principle of the TCL-Based Computing Cell

Fig. 6(a) shows the circuit structure of the proposed computing cell with 15 TCLs, TCL-0 to TCL-14, which realize lossless bit-column addition. The first MMU in each TCL is initialized from V_{DD}, and the last MMU drives the CNT. The 64 MMUs generate the 8-bit $\times$ 8-bit partial-product matrix for XIN[7:0] and W[7:0]. The 15 TCLs are coordinated by a local ten-phase generator shared by four CIM blocks. Instead of accumulating all partial products by a standard-cell-based conventional compressor tree, the TCL converts the bit-column sum into the number of full-swing transitions, which reduces routing complexity substantially.

Fig. 6(b) shows the circuit of the rising-edge delay cell (RDC) and the timing diagram of the local ten-phase generator. The local ten-phase generator consists of ten RDCs connected in cascade. When V_{rst} is asserted, all RDC outputs (Ph0–Ph9) are reset to low. After V_{rst} is deasserted, the rising edge propagates through the RDC chain and sequentially pulls Ph0–Ph9 high. The input-to-output delay, $\Delta t_{RDC,k}$, is controlled by V_{delay}. During $\Delta t_{RDC,k}$, Phk is high while Phk+ 1 is low. Among these phases, Ph0–Ph7 are used to sequentially activate the MMUs in the TCLs, Ph8 is used to enable the combine adder, and Ph9 is reserved for the subsequent adder tree. As a result, bit-column compression and the accumulation stages are executed in a deterministic temporal order within one clock cycle.

Fig. 6(c) shows the circuit and truth table of the MMU. Each MMU consists of a multiplication section and a multiplexing section. The multiplication section determines whether Path1 or Path2 is selected according to the values of Phk, XIN[m], and W[n], where k/m/n = 0,..., 7. Let P_{mn} = XIN[m] $\times$ W[n] denote the local partial product. Path1 is selected only when Phk = 1 and P_{mn} = 1. In this case, the MMU generates a transition by inverting the incoming signal. In all other cases, the MMU remains on Path2 and forwards the input signal to the next stage. Therefore, each partial product equal to "1" contributes one transition to a TCL, and the total number of such transitions in this TCL is recorded by the CNT as the local bit-column sum.

According to this TCL circuit structure, during $\Delta t_{RDC,k}$, MMU #k is newly enabled, MMU #0 to MMU #(k-1) keep the states selected in the previous phases, and MMU #(k + 1) to MMU #7 remain on Path2 because their phase signals are still low. Therefore, a new transition is generated and recorded when the partial product of the newly enabled MMU is "1"; otherwise, no new transition is produced.

Fig. 7 illustrates the TCL workflow by taking TCL-9 as an example, where the corresponding partial products {P_{07}, P_{17}, P_{27}, P_{36}, P_{45}, P_{54}, P_{63}, P_{72}} are {1, 1, 0, 0, 1, 0, 0, 1}. Transitions are generated during $\Delta t_{RDC,0}$, $\Delta t_{RDC,1}$, $\Delta t_{RDC,4}$, and $\Delta t_{RDC,7}$, whereas no transition is generated during $\Delta t_{RDC,2}$, $\Delta t_{RDC,3}$, $\Delta t_{RDC,5}$, or $\Delta t_{RDC,6}$. As a result, the dual-edge CNT records four transitions in total, corresponding to a bit-column sum of "4." During Ph0–Ph7, both Ph8 and Ph9 remain low, such that the downstream adder stages stay inactive until the TCL operation is completed.

In this way, the eight partial products in TCL-9 are losslessly compressed into a 4-bit count output, requiring only a single routing wire and thereby significantly alleviating routing congestion. Based on the same principle, the 15 TCLs together generate the 15 bit-column sums required for the 16-bit product reconstruction.

B. Timing Design of the Ten-Phase Generator Based on the Analysis of TCL

To ensure correct TCL operations, $\Delta t_{RDC,k}$ must be sufficiently large for MMU #k to complete the operation after Phk is pulled high and before Phk+ 1 is asserted. However, larger $\Delta t_{RDC,k}$ leads to reduced clock frequency and throughput. Therefore, the optimal $\Delta t_{RDC,k}$ must be carefully determined.

As shown in Fig. 6(c), when Phk = 0 or Phk = 1 with P_{mn} = 0, MMU #k operates as a transmission gate (TG), passing MIn to MOut, with delay t_{pass} dominated by the pass path of the TG. When Phk = 1 and P_{mn} = 1, MMU #k is configured as an inverter, generating a transition. In this case, the transition-generation delay t_{gen} is dominated by the control delay $t_{\&}$ in the multiplication section and the inverter delay t_{INV}. In general, t_{gen} is significantly larger than t_{pass}, so the minimal $\Delta t_{RDC,k}$ should be larger than t_{gen}.

As shown in Fig. 8, for a TCL with eight MMUs in unsigned INT8 mode, if Phk = 1 and P_{mn} = 1 at MMU #k, the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:18:21 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

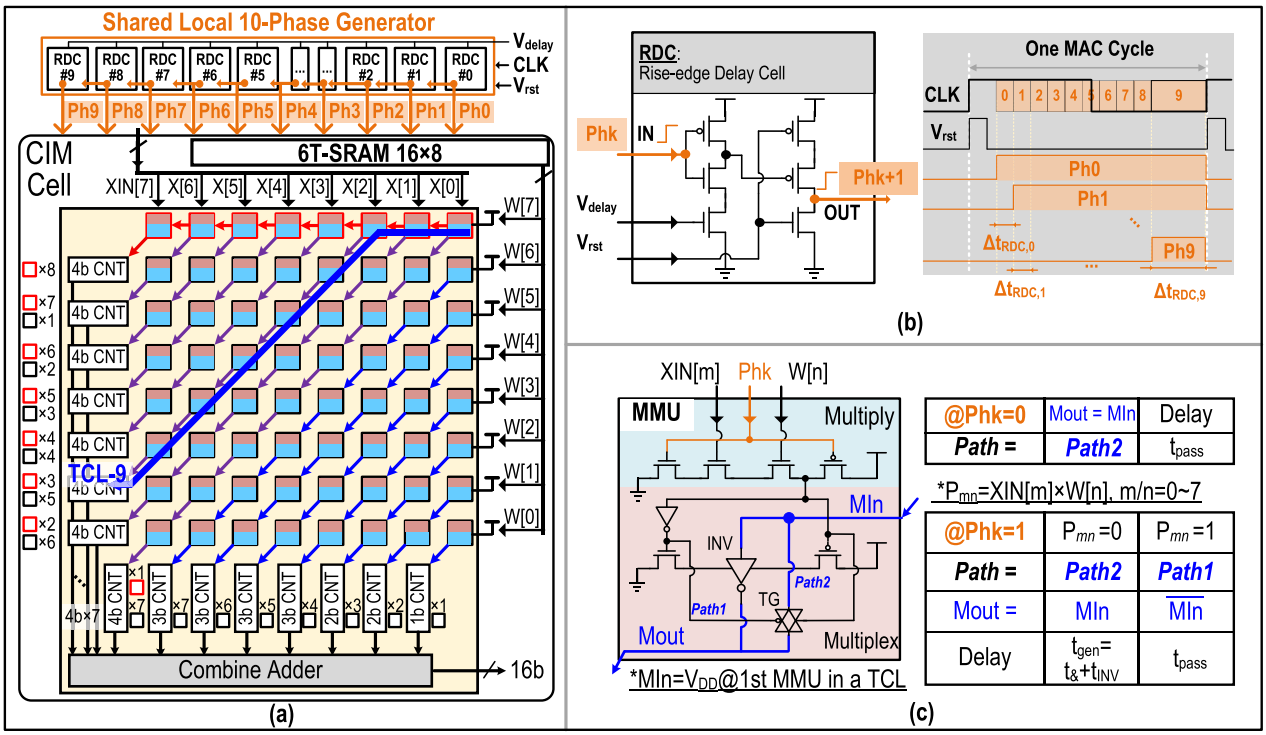


Fig. 6. (a) Structures of the local ten-phase generator and the TCL in the proposed TCL-based CIM macro. (b) Circuit of the RDC and timing diagram of the ten-phase generator. (c) Circuit of the MMU and its truth table of operation modes.

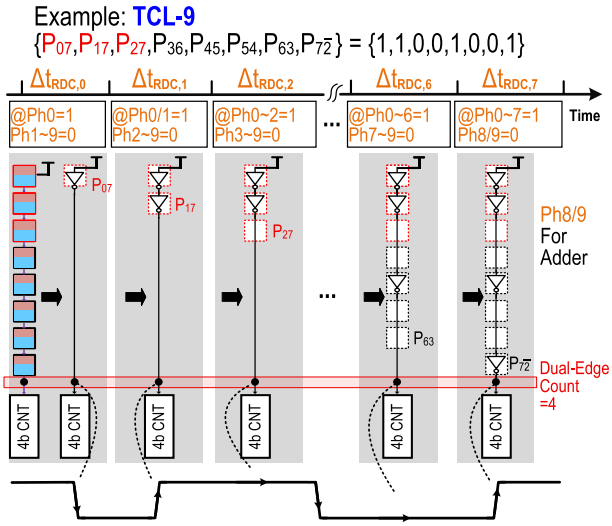


Fig. 7. Operation flow of the RDC, MMU, and CNT, taking TCL-9 as an example.

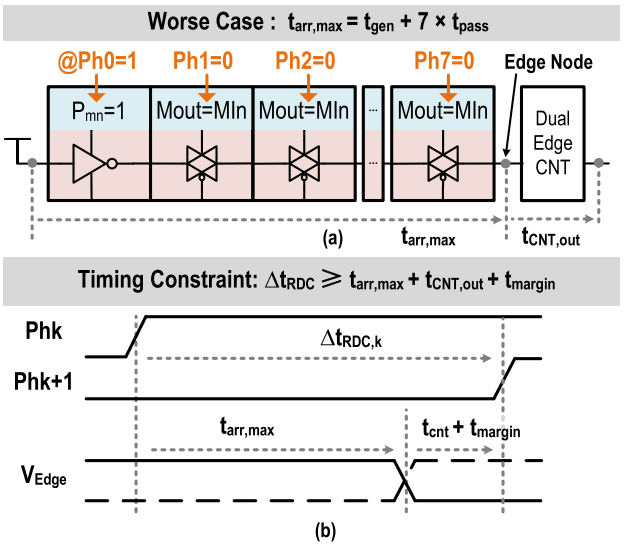


Fig. 8. Timing constraints of RDC. (a) Worst case propagation in unsigned mode when the first MMU is activated and (b) overall timing constraints.

generated transition has to propagate through the subsequent (7-k) MMUs with Phk+ 1 = Phk+ 2 =...= Ph7 = 0 before reaching the CNT input node. The corresponding transition arrival time at the CNT input node can, thus, be expressed as

$$t_{arr}^{(k)} \approx t_{gen} + (7-k) \times t_{pass}, \quad k = 0, 1, \ldots, 7. \tag{1}$$

The worst case occurs when k = 0, and the first MMU is selected as an inverter, yielding

$$t_{arr,max} \approx t_{gen} + 7 \times t_{pass}. \tag{2}$$

Assuming a uniform inter-phase spacing Δt_{RDC} across all stages, the worst case timing constraint is determined by the maximum transition propagation delay. To ensure reliable transition counting operation, Δt_{RDC} must be sufficiently greater than the worst case transition propagation delay $\Delta t_{MMU,wst}$, which must satisfy

$$\Delta t_{RDC} \geq \Delta t_{MMU,wst} = t_{arr,max} + t_{CNT,out} + t_{margin} \tag{3}$$

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:18:21 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

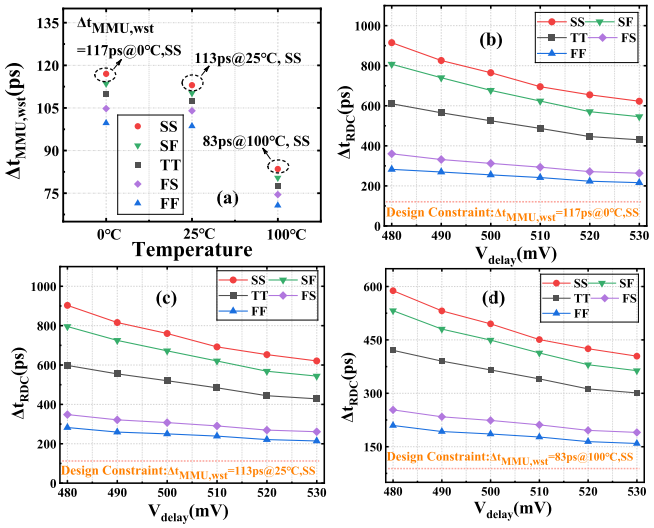


Fig. 9. (a) Post-layout simulation of $\Delta t_{MMU,wst}$ in unsigned mode under different process corners and at 0 °C, 25 °C, and 100 °C with a 0.9-V supply voltage. Post-layout simulations of Δt_{RDC} versus V_{delay} at a 0.9-V supply voltage and (b) 0 °C, (c) 25 °C, and (d) 100 °C, respectively.

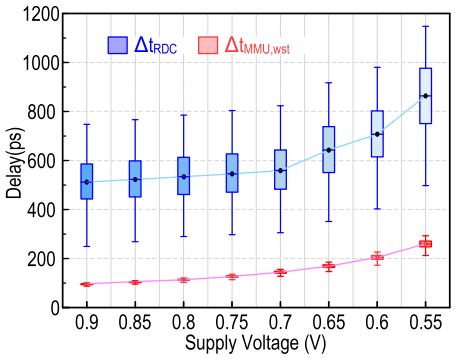


Fig. 10. Statistical box plots of simulated Δt_{RDC} and $\Delta t_{MMU,wst}$ in unsigned mode at 25 °C under different supply voltages.

where $t_{CNT,out}$ denotes the operation delay of a dual-edge CNT, t_{margin} denotes the timing margin for PVT variations.

These constraints indicate that the timing bottleneck of the TCL is the combined effect of transition generation in the local inversion path of MMU, the propagation delay of the subsequent cascaded MMUs operating as TGs, and the counting delay of the dual-edge CNT.

To determine the appropriate Δt_{RDC}, the $\Delta t_{MMU,wst}$ values under different process corners and at 0 °C, 25 °C, and 100 °C with a 0.9-V supply voltage are obtained from post-layout simulations, as shown in Fig. 9(a). The largest $\Delta t_{MMU,wst}$ occurs at the SS corner across all temperatures and is, therefore, used to set the RDC design constraints. Post-layout simulations of Δt_{RDC} versus V_{delay} at a 0.9-V supply voltage and 0 °C, 25 °C, and 100 °C are shown in Fig. 9(b)–(d), respectively. Δt_{RDC} values are consistently larger than $\Delta t_{MMU,wst}$ values, thereby guaranteeing reliable TCL operation. In addition, Δt_{RDC} can be tuned via V_{delay} to accommodate different design targets.

To evaluate the impact of supply voltage scaling and process variation on timing margins, statistical simulations with 10k samples are performed at supply voltages from 0.55 to 0.9 V,

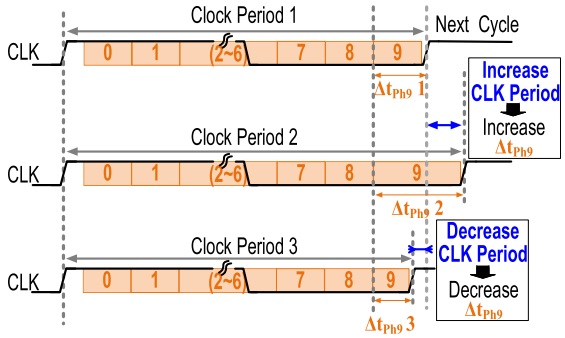


Fig. 11. CLK period determines the duration of Ph9.

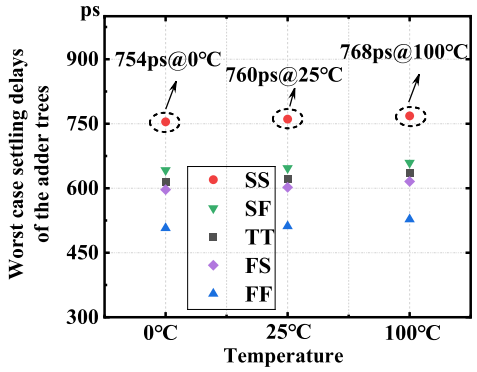


Fig. 12. Worst case settling delays of the adder trees under various temperature and process corners.

as shown in Fig. 10. The results demonstrate that $\Delta t_{MMU,wst}$ is consistently smaller than Δt_{RDC} across all supply voltages, ensuring robust TCL operation under voltage variations.

With a fixed Δt_{RDC}, the duration of Ph9 (Δt_{Ph9}) is determined by the clock period, as shown in Fig. 11, since only the valid V_{rst} in the next clock cycle resets the outputs of the RDCs to low. In other words, the Ph9 duration is determined by the clock period and Δt_{RDC}.

After Ph9 is asserted, the in-block adder tree is enabled, and its output further drives the following out-block adder tree before the final MAC result becomes stable.

The worst timing case of the accumulation path (the settling delay of the in-block adder tree + the settling delay of the out-block adder tree) occurs when the least significant bit (LSB) of the 16-b input of the in-block adder tree generates the MSB transition of the 22-b output of the out-block adder tree. We have constructed such input cases to accomplish the post-layout timing simulation under various temperatures and process corners. The results are shown in Fig. 12. Δt_{Ph9} is expected to be larger than the corresponding settling delays by adjusting the clock period.

C. Dual-Edge CNT Implementation

A dual-edge CNT is employed at the end of each TCL to convert the transitions into digital bit-column sums. Fig. 13 illustrates the proposed dual-edge CNTs supporting four output bit widths.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:18:21 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

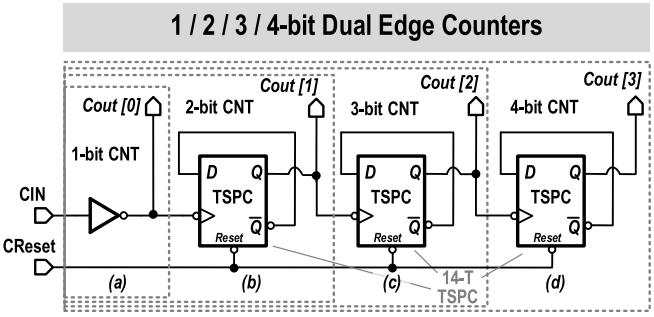


Fig. 13. Circuit of the proposed dual-edge CNTs: (a) 1-bit CNT, (b) 2-bit CNT, (c) 3-bit CNT, and (d) 4-bit CNT.

As TCL-0 contains only one MMU, a simple inverter suffices to produce a 1-bit count output, as shown in Fig. 13(a). TCL-1 and TCL-2 require 2-bit count outputs and are implemented using one inverter and one negative edge-triggered D flip-flop (DFF) with Q reset to zero. In particular, the output of the last MMU is inverted and used to drive the clock terminal of the DFF, while the $\overline{Q}$ output is fed back to the D input. In this configuration, the inverter output forms the LSB, while the Q output forms the MSB, as shown in Fig. 13(b).

TCL-3 to TCL-6 require 3-bit count outputs and are realized by one inverter followed by two negative-edge-triggered DFFs. The Q output of the first-stage DFF drives the clock terminal of the second-stage DFF, and the $\overline{Q}$ outputs of both DFFs are fed back to their respective D inputs. Accordingly, the two Q outputs correspond to the two MSBs, whereas the inverter output provides the LSB, as shown in Fig. 13(c). TCL-7 requires 4-bit count outputs. The 4-bit CNT is readily constructed by appending one additional DFF stage to the 3-bit CNT, as shown in Fig. 13(d).

All DFFs are implemented using a resettable 14-T true-single-phase-clock (TSPC) structure, which reduces the area and power consumption as compared to the conventional resettable static primary-secondary DFF based on TGs. The same function can be realized by positive-edge-triggered DFFs, but an additional inverter is required.

These DFFs are reset at the beginning of every MAC cycle, before the TCL transition-counting operation starts. The reset signal of the DFFs is denoted as CReset, as shown in Fig. 13, which is generated from the computation reset signal [V_{rst} shown in Fig. 6(b)] with the proper polarity. CReset resets Cout [3:1] to zero. The TCL input state is initialized to V_{DD}, such that the inverter-generated LSB Cout [0] is reset to zero directly.

In unsigned multiplication, TCL-8 to TCL-14 are symmetric to TCL-6 to TCL-0, so the CNTs in TCL-8 to TCL-14 require the same bit-width as in TCL-6 to TCL-0. In signed two's-complement multiplication, TCL-8 to TCL-14 are connected to a bus TCL for sign-bit extension, which will be described in detail in Section IV. In this case, TCL-8 to TCL-14 require 4-bit count outputs. Therefore, the CNTs in TCL-8 to TCL-14 are designed as 4-bit CNTs with the two MSBs reset to zero.

The CNT output represents the local sum of one bit column, and the subsequent combine adder merges these bit-column sums according to their binary weights to reconstruct the full-precision 16-bit product of XIN[7:0] × W[7:0]. In this way, the proposed dual-edge CNTs provide a compact digital interface between the TCL-based compression and the following product construction.

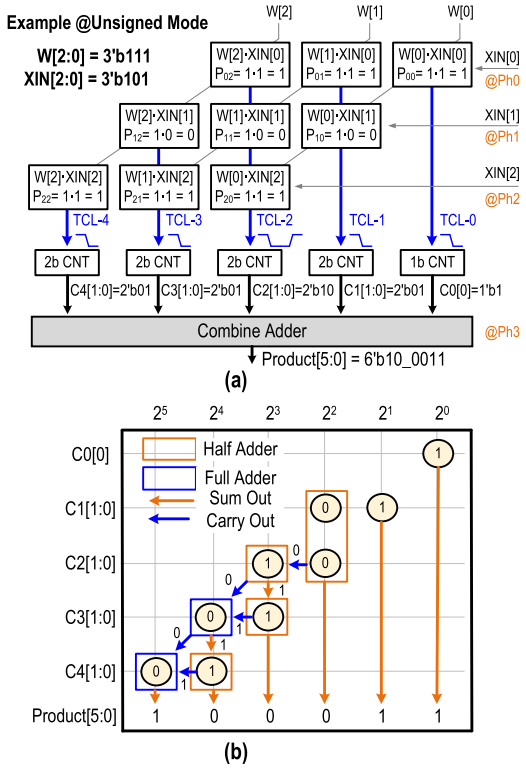


Fig. 14. Complete unsigned 3-b × 3-b multiplication using the TCL-based computing cell. (a) Partial product generation, bit-column transition counting, and CNT outputs, and (b) weighted-sum reconstruction in the combine adder.

D. End-to-End Multiplication and Weighted-Sum Reconstruction

To clarify the complete arithmetic operation of the proposed TCL-based computing cell, Fig. 14 presents an unsigned 3-bit × 3-bit multiplication example. P_{mn} = XIN[m] × W[n] denotes a local 1-bit partial product. TCL-k compresses all partial products satisfying. Since each partial product equal to "1" generates one transition when its MMU is activated, whereas a zero-valued partial product transparently passes the current TCL state, the CNT output of TCL-k is Ck = $\sum P_{mn}$, m+n = k. The full-precision product is then reconstructed by adding Ck with its original binary weight (bit position): Product [5:0] = $\sum_0^4 Ck \cdot 2^k$.

In the example of Fig. 14(a) with W[2:0] = 3'b111 and XIN[2:0] = 3'b101, TCL-0 to TCL-4 generate [1, 1, 2, 1, 1] transitions, respectively, and their CNT outputs are C0[0] = 1'b1, C1[1:0] = 2'b01, C2[1:0] = 2'b10, C3[1:0] = 2'b01, and C4[1:0] = 2'b01, respectively. As illustrated in Fig. 14(b), the combine adder aligns C0–C4 according to their bit positions and performs Product [5:0] = C0[0] ≪ 0+ C1[1:0] ≪ 1+ C2[1:0] ≪ 2+ C3[1:0] ≪ 3+ C4[1:0] ≪ 4 = 2'b10_0011. This example demonstrates that the TCLs losslessly compress

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:18:21 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

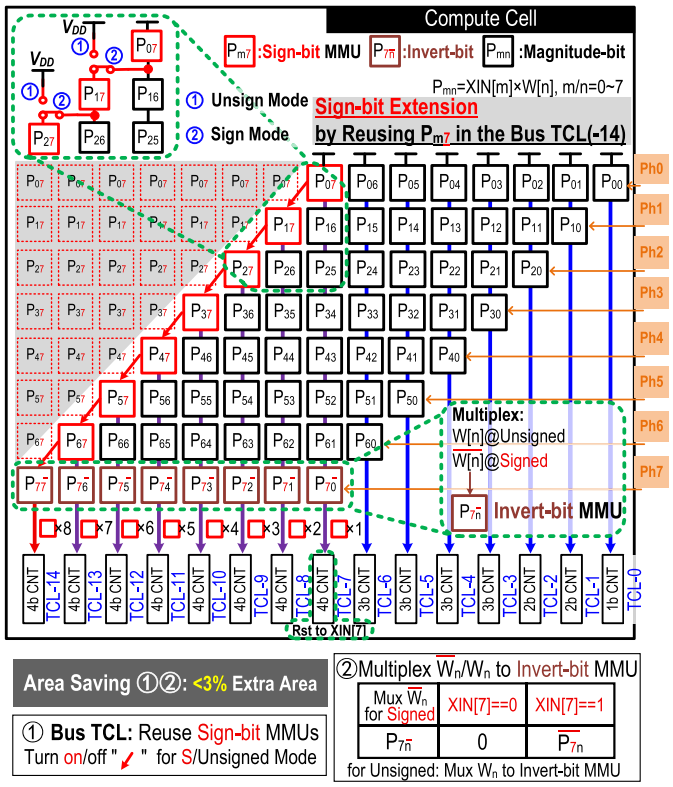


Fig. 15. Embedded sign-bit extension in bus TCL, the inversion processing, and the initialization correction in the computing cell.

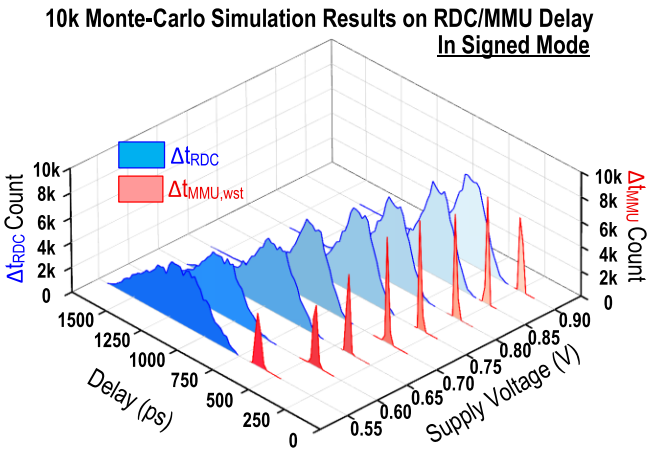


Fig. 16. 10k-sample Monte Carlo simulation results of Δt_{RDC} and $\Delta t_{MMU,wst}$ at 0.55–0.9-V supply voltages under 25 °C in signed mode. Fourteen t_{pass}'s are considered in $\Delta t_{MMU,wst}$.

the partial-product columns into digital counts and that the combine adder exactly reconstructs the full-precision product from the weighted CNT outputs.

IV. Embedded Sign-Bit in the Bus TCL and Stage-Wise-Enabled Accumulation

A. Embedded Sign-Bit Extension in the Bus TCL

Since XIN[7:0] and W[7:0] could be represented in unsigned format or signed two's-complement format, efficient support for signed multiplication without significantly increasing the area or introducing extra cycles is required. For 8-bit signed two's-complement operands, the MSB of weight is the sign bit, and the remaining 7 bits represent the magnitude. According to the two's-complement multiplication rule, sign-bit extension introduces additional sign-related partial products into the high-order columns of the partial-product matrix, as shown in Fig. 15. As a result, the original 64-term 8-bit × 8-bit partial-product matrix expands to 92 terms. In conventional digital schemes, these extra sign-bit extensions are processed either using additional full adders or in the post processing circuits after the local multiplication, which substantially increases area overhead or computation cycles.

To solve this problem, this work proposes embedded sign-bit extension in the bus TCL by reusing the sign-bit partial products already computed in the existing MMUs, as shown in Fig. 15. At first, the sign-bit-related MMUs in each row are connected to form a shared bus TCL. These MMUs are reused from TCL-7 to TCL-13 by broadcasting the MMU outputs. In this way, the presented bus TCL avoids substantial duplicated sign-related computations. For example, the output of the MMU corresponding to P_{07} is used to drive both the MMUs corresponding to P_{16} and P_{17}, respectively. The same reuse mechanism is applied to the following rows. Consequently, TCL-14 is formed by {P_{07}, P_{17}, P_{27}, P_{37}, P_{47}, P_{57}, P_{67}, P_{7_7}}, while TCL-7 to TCL-13 share the same sign-bit MMUs rather than duplicating them.

As shown in Fig. 15, the switching between unsigned and signed modes is realized using single-pole double-throw (SPDT) switches. In signed mode, the sign-bit MMUs are cascaded through the bus TCL, while in unsigned mode, the sign-bit bus path is deactivated, and the input of the first MMU in each TCL is tied to V_{DD}. Since TCL-7 to TCL-14 each contain eight MMUs after sign extension, a 4-bit CNT is used at the end of each of these TCLs. For signed two's-complement multiplication, the required inversion in the last row is realized by multiplexing the $\overline{W[n]}$ into the invert-bit MMUs, and the initialization correction is implemented by resetting the CNT in TCL-7 to XIN [7]. All these switches and multiplexers incur merely ~3% area overhead without additional computation cycles, thereby enabling efficient support for signed two's-complement multiplication.

The added seven SPDT switches introduce an additional seven TGs in the worst case timing path for signed two's-complement multiplication, according to the derivation in Section III-B. Therefore, the design constraint for Δt_{RDC} becomes $\Delta t_{RDC} \geq \Delta t_{MMU,wst} = t_{gen} + 14\times t_{pass} + t_{CNT,out} + t_{margin}$. The 10k-sample Monte Carlo simulation results for this signed mode circuit structure are shown in Fig. 16. Since $\Delta t_{MMU,wst}$ increases, V_{delay} is tuned to 480 mV to enlarge Δt_{RDC}, ensuring reliable TCL operation under process variations.

B. Stage-Wise-Enabled Accumulation

As shown in Fig. 6, the shared local ten-phase generator coordinates the TCL and the subsequent adder-tree accumulation within one computation cycle. Ph0–Ph7 sequentially

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:18:21 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

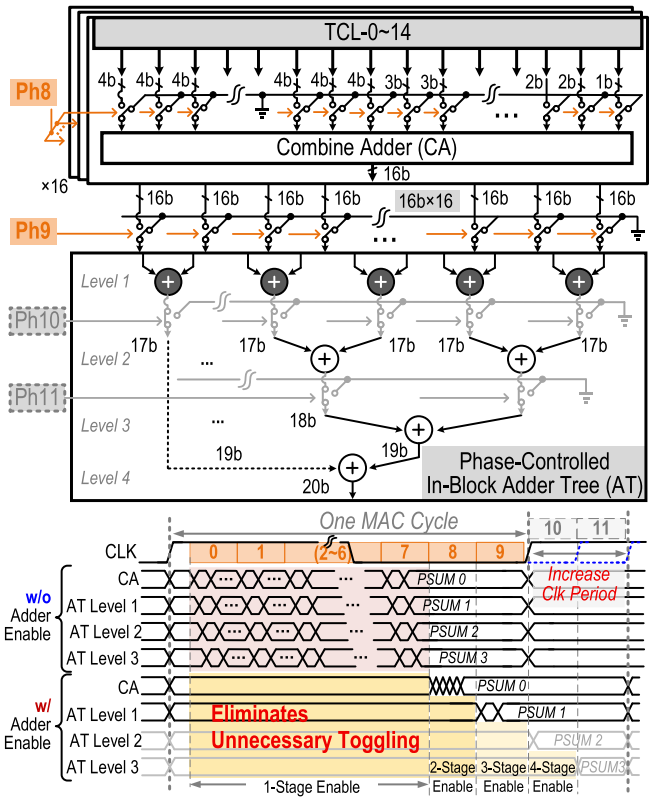


Fig. 17. Stage-wise enabling circuits for the combine adder and in-block adder tree.

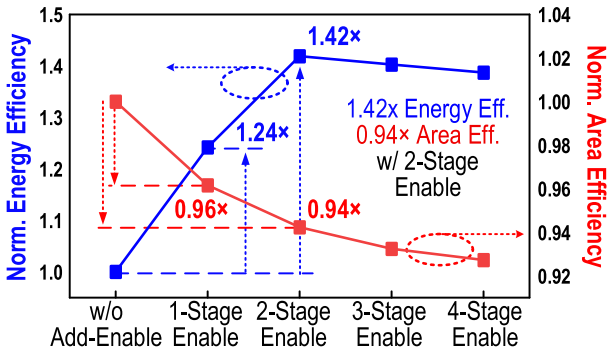


Fig. 18. Normalized energy and area efficiencies versus the number of enabling stages in adders.

activate the MMUs in each TCL to perform bit-column addition, and the CNT outputs are then combined by the combine adder to generate one 16-bit product per computing cell. The 16 products in one block are accumulated by the in-block adder tree to generate a 20-bit partial sum.

The CNT outputs become valid only after Ph7. During Ph0–Ph7, the rapidly varying CNT values correspond to intermediate bit-column sums. Direct propagation of these values to downstream adders induces significant spurious toggling, resulting in increased dynamic power consumption. This issue is further exacerbated in bit-parallel CIM due to the larger input widths of the adders.

To suppress this spurious toggling, the proposed design enables the accumulation path in a stage-wise manner. In the

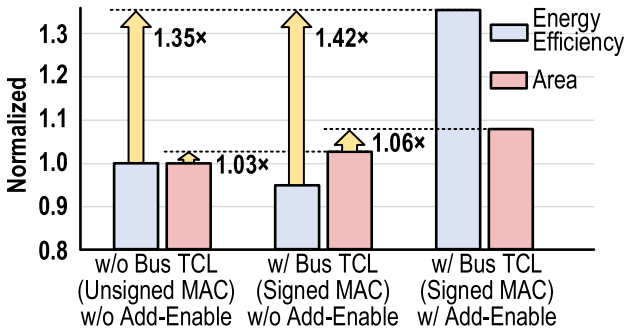


Fig. 19. Combined effect of the embedded sign-bit extension and the two-stage adder-enabling circuits.

one-stage-enable case, the combine adder (CA in Fig. 17) input is clamped to ground before Ph8 and is released only after the CNT outputs have settled, so that the combine adder processes only the final bit-column sums. In the two-stage-enable case, an additional isolation stage controlled by Ph9 is inserted at the input of the in-block adder tree (AT in Fig. 17), preventing the combine adder settling transitions from propagating into the in-block adder tree. As further illustrated in Fig. 17, three-stage and four-stage enabling can suppress toggling in deeper levels of the adder tree by introducing additional phases, such as Ph10 and Ph11. However, these extra enabling stages increase the clock period and add more switches in the accumulation path. The post-layout simulation on area and energy efficiency versus the number of enabling stages is performed, as shown in Fig. 18. As a tradeoff, the two-stage-enable scheme is adopted in this work, with one enabling stage before the combine adder and one before the in-block adder tree. It achieves a 1.42× energy-efficiency improvement with only a 6% area-efficiency degradation, providing the best energy-area tradeoff among the evaluated configurations.

To demonstrate the combined effect of the embedded sign-bit extension and the two-stage adder-enabling circuits, the normalized energy efficiency and area overhead are evaluated, as shown in Fig. 19. The unsigned TCL-based computing cell without bus TCL connection and adder-enabling circuits is referred to as the baseline. Incorporating the switches and multiplexers incurs a 3% area overhead and results in a 5% degeneration in energy efficiency. With additional switches for adder enabling, a further 6% area overhead is introduced while the energy efficiency is improved by 1.42×. Compared with the baseline, the integration of sign-bit extension and adder enabling achieves a 1.35× energy efficiency improvement with only 9% area overhead, demonstrating a favorable energy-area tradeoff.

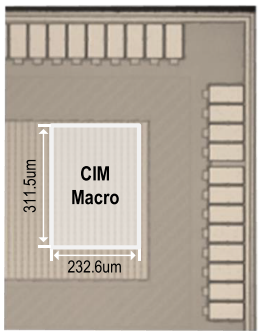
V. Experimental Results

A. Measurement Results

Fig. 20 presents the die photograph and summarizes the chip measurement results. This CIM macro is fabricated in 28-nm CMOS technology and occupies 0.072 mm^2 (232.6 × 311.5 μm), with a memory capacity of 32 kb and an operating voltage range of 0.55–0.9 V.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:18:21 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



CHIP SUMMARY		
Technology	28nm CMOS	
Cell Structure	Compact 6T+TCLs	
Macro Size	32Kb	
Macro Area (mm^2)	0.072 (232.6um×311.5um)	
Input Precision (bit)	INT8	BF16
Weight Precision (bit)	INT8	BF16
Number of IN Channels (N_{ACCU})	64	64
Output Precision (bit)	INT22	FP32
Supply Voltage (V)	0.55-0.9	
Output Ratio	1	
Access Time (ns)	4.1 @ 0.9V	
[1]Average Energy Efficiency (TOPS/W) (TFLOPS/W)	24.76-65.26	17.32-48.17
[2]Peak Energy Efficiency (TOPS/W) (TFLOPS/W)	40.74-106.85	28.49-77.68
Area Efficiency (TOPS/mm^2) (TFLOPS/mm^2)	1.73	[3] 1.24
Memory Density (Kb/mm^2)	442.09	
Inference Accuracy [4]ResNet50 @ImageNet1k_V2	80.10%	80.19%
Inference Accuracy [5]ViT-B @ImageNet1k_V2	79.50%	81.05%

[1] Measured under 50% input sparsity, 50% input toggle rate, 50% weight sparsity
[2] Measured under 90% input sparsity, 10% input toggle rate, 50% weight sparsity
[3] Including exponent computation circuits.
[4] Software baseline (FP32) is 80.34%.
[5] Software baseline (FP32) is 81.07%.

Fig. 20. Die photograph and chip performance summary.

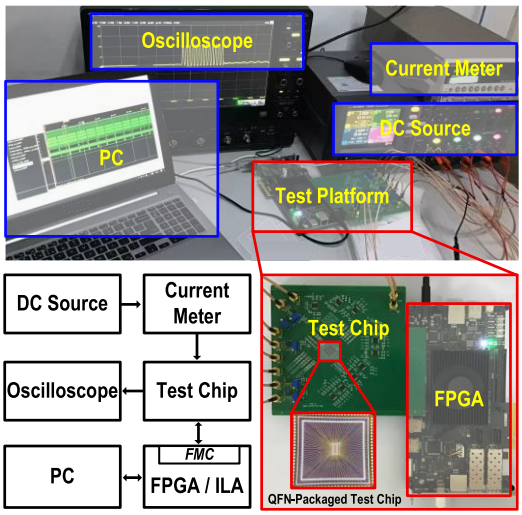


Fig. 22. Test platform.

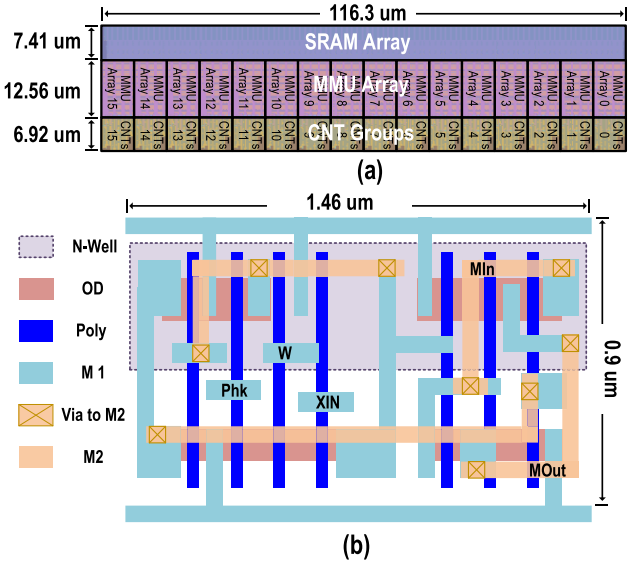


Fig. 21. (a) Rotated layout of the SRAM array, MMU array, and CNT groups in a computing cell and (b) layout sketch and physical dimensions of a single MMU.

Fig. 21(a) shows the layout snapshot with the 6T-SRAM array, the MMU arrays, and CNT groups in one computing cell. Fig. 21(b) presents the layout sketch and physical dimensions of a single MMU, where M1 and M2 layers implement local interconnects, and only using M2 could support the routing within the MMU array.

The local computing region is height-matched to the SRAM rows, corresponding to approximately 7.27 μm (116.3 μm/16). A direct 8-b $\times$ 8-b bit-parallel implementation would need to route 64 one-bit partial-product signals within this height, leading to an average signal pitch of approximately 7.27 μm/64 = 0.113 μm. This estimate only accounts for the partial-product input wires and does not include sign-extension terms, control signals, or via enclosure. By compressing the 64 partial products into 15 weighted bit-column signals, the proposed TCL structure relaxes the required pitch to 7.27 μm/15 = 0.48 μm, providing a 4.27$\times$ relaxation in the local routing demand.

Fig. 22 shows the measurement platform. An FPGA is used to send the weights and feature maps to the chip and to receive the computation results. A current meter measures the current consumption during computation. As the proposed TCL performs lossless bit-column compression, the output ratio remains 1, preserving full-precision outputs. The corresponding INT8 and BF16 inference accuracies on ResNet50@ImageNet1k_V2 are 80.10% and 80.19%, respectively, with accuracy losses of 0.24% and 0.15%, respectively, relative to the FP32 baseline of 80.34%. For ViT-B@ImageNet1k_V2, the INT8 and BF16 accuracies are 79.50% and 81.05%, with corresponding losses of 1.57% and 0.02%, respectively, relative to the FP32 baseline of 81.07%.

As shown in Fig. 23, the energy efficiency of the proposed CIM macro is measured under different input sparsity and toggle-rate conditions across various supply voltages, with the weight sparsity fixed at 50%. Since the TCL performs sparsity-adaptive bit-column addition, higher input sparsity and lower input toggle rate lead to fewer transition events, thereby leading to higher energy efficiency. The energy efficiency increases as the supply voltage scales down. In signed INT8 mode, a peak energy efficiency of 106.85 TOPS/W is achieved at 0.55 V under 90% input sparsity and 10% input toggle rate. In unsigned mode, the throughput could be further improved by tuning V_{delay} to reduce Δt_{RDC}, since the MMU with seven fewer TGs in the critical timing path works faster in unsigned mode.

Fig. 24 shows the area and measured power breakdown of the proposed TCL-based CIM macro under a 0.9-V supply voltage, with 50% weight sparsity and 50% input sparsity in INT8 mode. The MMU array and the adder tree occupy the largest area fractions, accounting for 32.18% and 29.38%, respectively. The adder tree and the CNT dominate the power consumption, contributing 35.37% and 25.64%, respectively. Under identical conditions, post-layout simulations show reset operation accounts for 6.6% of the total power consumption.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:18:21 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

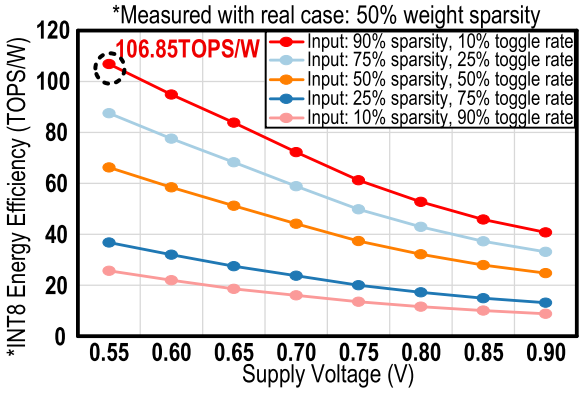


Fig. 23. Energy efficiency under input toggle rate/sparsity and voltage (weight sparsity @50%).

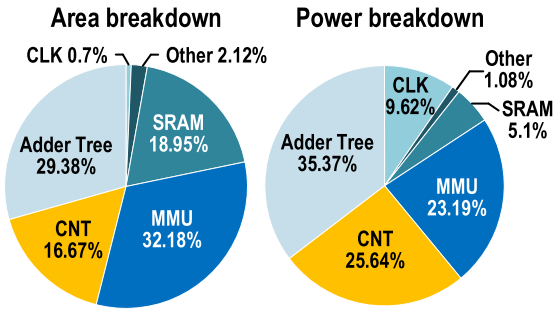


Fig. 24. Area and power breakdown of the proposed TCL CIM Macro.

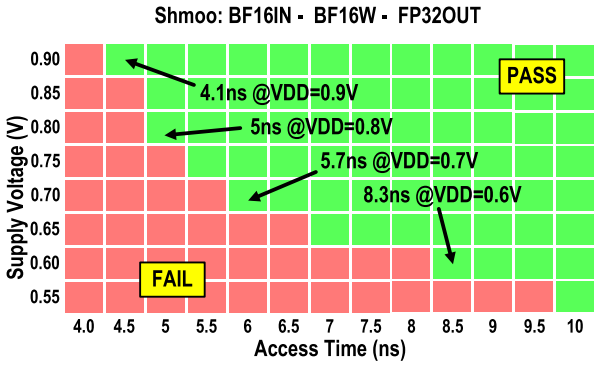


Fig. 25. Measured shmoo plot showing voltage versus working frequency.

Because the TCLs locally compress the partial-product matrix before the accumulation stage, the fan-in and toggling of the downstream adder tree are both reduced. In addition, the TSPC-based CNT incurs only a small area overhead.

Fig. 25 presents the measured shmoo plot for the BF16IN-BF16W-FP32OUT MAC operation. In BF16 mode, exponent pre-alignment, mantissa MAC, and exponent post-processing are executed in a pipelined manner. Because the mantissa MAC in BF16 mode reuses the same integer MAC core as in INT8 mode, Fig. 25 also reflects the voltage-frequency characteristics of INT8 MAC operation. The measured access time is 4.1 ns at 0.9 V, 5.0 ns at 0.8 V, 5.7 ns at 0.7 V, and 8.3 ns at 0.6 V.

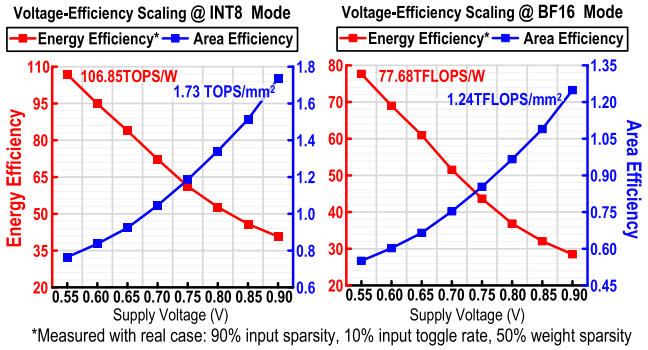


Fig. 26. Voltage-efficiency scaling curves. The energy efficiency and area efficiency in INT8 and BF16 modes.

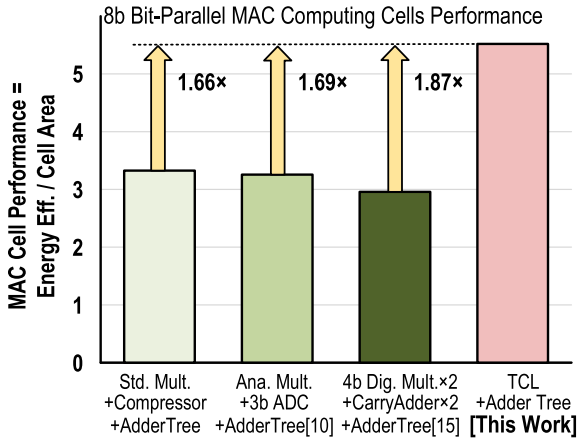


Fig. 27. MAC cell performance of various computing cell structures for 8b bit-parallel MAC.

Fig. 26 shows the energy-efficiency scaling curves versus supply voltage in BF16 and INT8 modes, respectively, measured under realistic activity conditions with 90% input sparsity, 10% input toggle rate, and 50% weight sparsity. In BF16 mode, the peak energy efficiency reaches 77.68 TFLOPS/W at 0.55 V, and the peak area efficiency reaches 1.24 TFLOPS/mm^2 at 0.9 V. In INT8 mode, the peak energy efficiency and area efficiency are 106.85 TOPS/W at 0.55 V and 1.73 TOPS/mm^2 at 0.9 V, respectively.

B. Comparison With State-of-the-Art Works

To enable comparison with previous bit-parallel works, the normalized MAC cell performance (the energy efficiency over cell area) of various 8-bit bit-parallel designs is calculated, as shown in Fig. 27. A standard digital implementation consisting of a multiplier, a compressor tree, and an adder tree is used as the baseline. The proposed TCL-based MAC cell is compared with an analog implementation employing analog multiplication, the 3-bit ADC, and an adder tree [10], and with a digital-CIM implementation using two 4-bit bit-parallel MAC cells, two ripple-carry adders, and an adder tree [15]. The comparison targets signed 8-bit two's-complement MAC operations. Therefore, the sign-bit extension is implemented in the baseline digital design, the proposed TCL CIM is configured in signed mode, and [10] and [15] handle sign

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:18:21 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE I
COMPARISON WITH STATE-OF-THE-ART WORKS

	Signed Bit-Parallel CIM					Signed Bit-Serial CIM		
	This Work		ISSCC 2025[15]		ISSCC 2023[5]	ISSCC 2025[42]	ISSCC 2025[11]	
Technology	28nm		28nm		22nm	28nm	28nm	
CIM Type	Digital		Digital		Hybrid	Hybrid	Digital	
Supply Voltage	0.55-0.9		0.55-0.9		0.6-0.9	0.6-0.9	0.55-0.9	
Macro Size	32Kb		32Kb		832Kb	64Kb	64Kb	
MAC Cell Count	256		64		256	512	256	
Area (mm^2)	0.072		0.043		18	0.108	0.136	
Memory Density (Kb/mm^2)	442.09		744.19		46.22	580	470.59	
Input Precision (bit)	INT8	BF16	INT8	BF16	BF16	INT8	INT8	BF16
Weight Precision (bit)	INT8	BF16	INT8	BF16	BF16	INT8	INT8	BF16
Output Precision (bit)	INT22	FP32	INT22	-	FP32	INT21-25	INT20	FP32
Number of IN Channels (N_{ACCU})	64		64		128	32-512	16	
Access Time (ns)	4.1		2.1	5.3	6.4	12	4.8	5.4
INT8 Energy Eff. (TOPS/W)	**40.74 - 106.85** **1*		36.5 - 104		-	21.04-67.8	25.47 - 90.15	
BF16 Energy Eff. (TFLOPS/W)	**28.49 - 77.68** **1*		25.6 - 75.8		62.14 - 70.21	-	17.83 - 62.84	
INT8 Area Eff. ($TOPS/mm^2$)	**1.73**		1.42 **2*		-	0.37-1.57	0.784	
BF16 Area Eff. ($TFLOPS/mm^2$)	**1.24** **3*		0.835 **3*		0.069	-	0.697	
FoM1 **4*	**4066.71**		3248.96		-	2661.15	1413.55	
FoM2 **5*	**96.32**		63.29		4.84	-	43.80	
Inference Accuracy Loss	0.02% BF16×BF16 ViT@ImageNet		0.1% BF16×BF16, VGG19@CIFAR100		0.43% BF16×BF16 ResNet18@ImageNet	1.75% INT8×INT8 ViT@ImageNet	0.016% BF16×BF16 ViT@ImageNet	

**1 : Measured under 90% input sparsity, 10% toggle rate, 50% weight sparsity; *2: Recalculated: (2 Ops × MAC Cell Count) / Access Time / Area; *3: Including exponent computation circuits; *4 : FoM1 = OUT Precision × TOPS/W × TOPS/mm² ; *5: FoM2 = TFLOPS/W × TFLOPS/mm² .*

bits outside and within the multipliers, respectively. The proposed TCL CIM improves the MAC cell FoM by 1.66×, 1.69×, and 1.87× over the baseline digital design, the analog implementation [10], and the 4-bit digital implementation [15], respectively.

Table I shows the comparison of the presented CIM macro with previous state-of-the-art works. Fabricated in 28-nm CMOS, the proposed macro is a 32-kb digital signed bit-parallel CIM macro occupying 0.072 mm^2 and supporting 64 input channels. In INT8 mode, it achieves a 4.1-ns access time at 0.9 V, a measured energy-efficiency range of 40.74–106.85 TOPS/W across 0.55–0.9 V, and a peak area efficiency of 1.73 $TOPS/mm^2$. In BF16 mode, the mantissa MAC reuses the INT8 MAC core and therefore exhibits the same 4.1-ns access time at 0.9 V. The corresponding measured energy-efficiency range is 28.49–77.68 TFLOPS/W, and the peak area efficiency is 1.24 $TFLOPS/mm^2$, including the exponent-processing circuits. Under the weight-stationary dataflow, adding post-layout simulated BF16 weight conversion energy to the measured 0.9 V BF16 CIM macro energy degrades the energy efficiencies to 27.50 and 27.67 TFLOPS/W for one-time inference on ResNet-50 and ViT-B, respectively. Using macro-level metrics defined as FoM1 = Output Precision × TOPS/W × $TOPS/mm^2$ for INT8 mode and FoM2 = TFLOPS/W × $TFLOPS/mm^2$ for BF16 mode [8], [12], this work achieves the highest macro-level FoMs among the compared macros in both modes.

VI. CONCLUSION

This article presented a 28-nm bit-parallel digital SRAM CIM macro that combines TCL-based lossless bit-column compression, bus-TCL-based embedded sign-bit extension, and stage-wise-enabled accumulation to address routing congestion, signed-MAC overhead, and spurious adder toggling in bit-parallel CIM. The TCL compresses partial products in a bit column to reduce interconnect density without computation loss, while the bus TCL directly embeds sign handling, eliminating explicit sign-extension circuitry. The stage-wise accumulation further suppresses unnecessary toggling in the adder tree, enhancing overall energy efficiency. Fabricated in 28-nm CMOS, the 32-kb macro occupies 0.072 mm^2, supports 64 input channels, achieves a 4.1-ns access time at 0.9 V, and delivers peak efficiencies of 106.85 TOPS/W for INT8 MAC and 77.68 TFLOPS/W for BF16 MAC. These results demonstrate that transition counting and phase-scheduled accumulation provide an effective path toward accurate, compact, and energy-efficient bit-parallel CIM macros for edge-AI devices.

REFERENCES

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[2] T. B. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.

[3] A. Sebastian, M. L. Gallo, R. Khaddam-Aljameh, and E. Eleftheriou, "Memory devices and applications for in-memory computing," *Nature Nanotechnol.*, vol. 15, no. 7, pp. 529–544, Jul. 2020.

[4] A. Guo et al., "A 28-nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6T-SRAM computing-in-memory macro for floating-point CNNs," *IEEE J. Solid-State Circuits*, vol. 59, no. 9, pp. 3032–3044, Sep. 2024.

[5] P.-C. Wu et al., "A 22 nm 832Kb hybrid-domain floating-point SRAM in-memory-compute macro with 16.2-70.2 TFLOPS/W for high-accuracy AI-edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 126–128.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:18:21 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

[6] P. Chen et al., "7.8 A 22 nm delta-sigma computing-in-memory (ΔΣCIM) SRAM macro with near-zero-mean outputs and LSB-first ADCs achieving 21.38TOPS/W for 8b-MAC edge AI processing," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 140–142.

[7] A. Guo et al., "34.3 A 22 nm 64kb lightning-like hybrid computing-in-memory macro with a compressed adder tree and analog-storage quantizers for transformer and CNNs," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 570–572.

[8] W.-S. Khwa et al., "34.2 A 16 nm 96Kb integer/floating-point dual-mode-gain-cell-computing-in-memory macro achieving 73.3-163.3TOPS/W and 33.2-91.2TFLOPS/W for AI-edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 568–570.

[9] H. Fujiwara et al., "34.4 A 3 nm, 32.5TOPS/W, 55.0TOPS/mm^2 and 3.78Mb/mm^2 fully-digital compute-in-memory macro supporting INT12× INT12 with a parallel-MAC architecture and foundry 6T-SRAM bit cell," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 572–574.

[10] Y. Yuan et al., "34.6 A 28 nm 72.12TFLOPS/W hybrid-domain outer-product based floating-point SRAM computing-in-memory macro with logarithm bit-width residual ADC," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 576–578.

[11] X. Wang et al., "14.3 A 28 nm 17.83-to-62.84TFLOPS/W broadcast-alignment floating-point CIM macro with non-two's-complement MAC for CNNs and transformers," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2025, pp. 254–256.

[12] W.-S. Khwa et al., "14.2 A 16 nm 216kb, 188.4TOPS/W and 133.5TFLOPS/W microscaling multi-mode gain-cell CIM macro edge-AI devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2025, pp. 1–3.

[13] Z. Yue et al., "14.4 A 51.6TFLOPs/W full-datapath CIM macro approaching sparsity bound and <2^{-30} loss for compound AI," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2025, pp. 1–3.

[14] H. Mori et al., "A 3 nm 125 Tops/W-29 TFLOPS/W, 90 TOPS/mm^2-17 TFLOPS/mm^2 SRAM-based INT8 and FP16 digital-CIM compiler with multi-weight update/cycle," in *Proc. Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2025, pp. 1–3.

[15] Y. Yuan et al., "14.5 A 28 nm 192.3TFLOPS/W accurate/approximate dual-mode-transpose digital 6T-SRAM CIM macro for floating-point edge training and inference," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2025, pp. 258–260.

[16] S. He et al., "A 22-nm 109.3-to-249.5-TFLOPS/W outlier-aware floating-point SRAM compute-in-memory macro for large language models," *IEEE J. Solid-State Circuits*, vol. 61, no. 2, pp. 736–749, Feb. 2026.

[17] R. Guo et al., "A 28 nm 4170-TFLOPS/W/b and 195-TFLOPS/mm^2/b multiply-free fully-digital floating-point compute-in-memory macro with Mitchell's approximation," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Honolulu, HI, USA, Jun. 2024, pp. 1–2.

[18] X. Si et al., "A local computing cell and 6T SRAM-based computing-in-memory macro with 8-b MAC operation for edge AI chips," *IEEE J. Solid-State Circuits*, vol. 56, no. 9, pp. 2817–2831, Sep. 2021.

[19] J.-W. Su et al., "Two-way transpose multibit 6T SRAM computing-in-memory macro for inference-training AI edge chips," *IEEE J. Solid-State Circuits*, vol. 57, no. 2, pp. 609–624, Feb. 2022.

[20] Y.-D. Chih et al., "16.4 An 89TOPS/W and 16.3TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22 nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 252–254.

[21] H. Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm^2 fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[22] Y. Wang et al., "ETCIM: Error-tolerant digital CIM processor with redundancy-free hard error repair and run-time soft error correction," *IEEE J. Solid-State Circuits*, vol. 61, no. 2, pp. 763–776, Feb. 2026.

[23] Y. He et al., "7.3 A 28 nm 38-to-102-TOPS/W 8b multiply-less approximate digital SRAM compute-in-memory macro for neural-network inference," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 130–132.

[24] S.-E. Hsieh et al., "7.6 A 70.85-86.27TOPS/W PVT-insensitive 8b word-wise ACIM with post-processing relaxation," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 136–138.

[25] Z. Yue et al., "7.7 CV-CIM: A 28 nm XOR-derived similarity-aware computation-in-memory for cost-volume construction," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 1–3.

[26] R. Guo et al., "20.2 A 28 nm 74.34TFLOPS/W BF16 heterogenous CIM-based accelerator exploiting denoising-similarity for diffusion models," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 362–364.

[27] F. Tu et al., "A 28 nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[28] B. Yan et al., "A 1.041-Mb/mm^2 27.38-TOPS/W signed-INT8 dynamic logic-based ADC-less SRAM compute-in-memory macro in 28 nm with reconfigurable bitwise operation for AI and embedded applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 188–190.

[29] J. Yue et al., "A 28 nm 16.9-300TOPS/W computing-in-memory processor supporting floating-point NN inference/training with intensive-CIM sparse-digital architecture," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 1–3.

[30] H. Mori et al., "A 4nm 6163-TOPS/W/b 4790 – TOPS/mm^2/b SRAM based digital-computing-in-memory macro supporting bit-width flexibility and simultaneous MAC and weight update," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 132–134.

[31] C.-F. Lee et al., "A 12 nm 121-TOPS/W 41.6-TOPS/mm^2 all digital full precision SRAM-based compute-in-memory with configurable bit-width for AI edge applications," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2022, pp. 24–25.

[32] S. Hong et al., "Dyamond: Compact and efficient 1T1C DRAM IMC accelerator with bit column addition for memory-intensive AI," *IEEE J. Solid-State Circuits*, vol. 60, no. 4, pp. 1299–1310, Apr. 2025.

[33] S. Kim et al., "DynaPlasia: An eDRAM in-memory computing-based reconfigurable spatial accelerator with triple-mode cell," *IEEE J. Solid-State Circuits*, vol. 59, no. 1, pp. 102–115, Jan. 2024.

[34] T.-H. Wen et al., "34.8 A 22 nm 16Mb floating-point ReRAM compute-in-memory macro with 31.2TFLOPS/W for AI edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 580–582.

[35] D.-Q. You et al., "14.1 A 22nm 104.5TOPS/W μ-NMC-Δ-IMC heterogeneous STT-MRAM CIM macro for noise-tolerant Bayesian neural networks," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2025, pp. 250–251.

[36] Y.-C. Chiu et al., "A 22 nm 4Mb STT-MRAM data-encrypted near-memory computation macro with a 192GB/s read-and-decryption bandwidth and 25.1-55.1TOPS/W 8b MAC for AI operations," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2022, pp. 178–180.

[37] L. Wang et al., "34.9 A flash-SRAM-ADC-fused plastic computing-in-memory macro for learning in neural networks in a standard 14nm FinFET process," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 582–584.

[38] W.-S. Khwa et al., "A 40-nm, 2M-cell, 8b-precision, hybrid SLC-MLC PCM computing-in-memory macro with 20.5-65.0TOPS/W for tiny-AI edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2022, pp. 180–182.

[39] X. Wang et al., "30.1 A 28 nm 127.54TFLOPS/W MXFP6 and 117.42TFLOPS/W MXFP8 compute-in-memory macro with adaptive-preserved-bit-width and serial-dual-bit-sliding schemes," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2026, pp. 512–514.

[40] Y. Wang et al., "A 28 nm 83.23TFLOPS/W POSIT-based compute-in-memory macro for high-accuracy AI applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 566–567.

[41] H. Wu et al., "A 28-nm 88.3-TFLOPS/W POSIT-approximate-calculation-based digital computing-in-memory macro incorporating final-cycle fusion and joint skipping," *IEEE J. Solid-State Circuits*, vol. 61, no. 4, pp. 1682–1696, Apr. 2026.

[42] X. Chen et al., "14.6 A 28 nm 64kb bit-rotated hybrid-CIM macro with an embedded sign-bit-processing array and a multi-bit-fusion dual-granularity cooperative quantizer," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2025, pp. 260–262.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:18:21 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

[43] Y. Wang et al., "TensorCIM: Digital computing-in-memory tensor processor with multichip-module-based architecture for beyond-NN acceleration," *IEEE J. Solid-State Circuits*, vol. 60, no. 2, pp. 734–747, Feb. 2025.
[44] Y. Ju, Y. Wei, and J. Gu, "A 65 nm general-purpose compute-in-memory processor supporting both general programming and deep learning tasks," *IEEE J. Solid-State Circuits*, vol. 60, no. 4, pp. 1500–1511, Apr. 2025.
[45] R. Guo et al., "A 28-nm 28.8-TOPS/W attention-based NN processor with correlative CIM ring architecture and dataflow-reshaped digital-assisted CIM array," *IEEE J. Solid-State Circuits*, vol. 60, no. 1, pp. 332–346, Jan. 2025.
[46] Y. Dong et al., "A heterogeneous TinyML SoC with energy-event-performance-aware management and compute-in-memory two-stage event-driven wakeup," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Honolulu, HI, USA, Jun. 2024, pp. 1–2.
[47] F. Tu et al., "MulTCIM: Digital computing-in-memory-based multimodal transformer accelerator with attention-token-bit hybrid sparsity," *IEEE J. Solid-State Circuits*, vol. 59, no. 1, pp. 90–101, Jan. 2024.
[48] R. Guo et al., "CIMFormer: A systolic CIM-array-based transformer accelerator with token-pruning-aware attention reformulating and principal possibility gathering," *IEEE J. Solid-State Circuits*, vol. 59, no. 10, pp. 3317–3329, Oct. 2024.
[49] H. Diao et al., "A computing-in-memory engine supporting one-shot floating-point NN inference and on-device fine-tuning for edge AI," *IEEE J. Solid-State Circuits*, vol. 60, no. 9, pp. 3403–3415, Sep. 2025.



Yunlong Liu (Graduate Student Member, IEEE) received the M.S. degree in electronic science from Anhui University, Hefei, China, in 2024. He is currently pursuing the Ph.D. degree with Xidian University, Xi'an, China.

His research focuses on CIM macros, particularly CIM's application in computing systems for image super-resolution.



Lichen Feng received the B.S. and Ph.D. degrees in microelectronics from Xi'an Jiaotong University, Xi'an, China, in 2014 and 2020, respectively.

Since 2020, he has been a Huashan Associated Professor with the School of Integrated Circuits, Xidian University, Xi'an. His research interests include intelligent processing chips near sensors, including algorithm-circuit codesign for lightweight machine learning, neuromorphic computing, and analog computing, etc.



Lin Wu received the B.S. degree in microelectronics from the School of Integrated Circuits, Xidian University, Xi'an, China, in 2024, where he is currently pursuing the M.S. degree in integrated circuits science and engineering with the School of Integrated Circuits.

His research interests include analog computing near sensors and compact column ADCs for CIM arrays.



Dong Li received the B.S. degree in electronic information science and technology from Southwest Jiao Tong University, Chengdu, China, in 2013, and the M.S. and Ph.D. degrees in microelectronics from Xidian University, Xi'an, China, in 2018 and 2021, respectively.

Since 2021, he has been a Huashan Associated Professor with the School of Integrated Circuits, Xidian University. His research interests include digital signal processing and system architecture for LiDAR.



Zhangming Zhu received the B.S., M.S., and Ph.D. degrees in microelectronics from Xidian University, Xi'an, China, in 2000, 2003, and 2004, respectively.

Since 2009, he has been a Professor with the School of Integrated Circuits, Xidian University. He has published more than 300 articles in IEEE journals. His current research interests include analog, mixed-signal, integrated circuits, and systems.

Dr. Zhu served as an Associate Editor for IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—II: EXPRESS BRIEFS.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:18:21 UTC from IEEE Xplore. Restrictions apply.